Web-Technologien UE - Assignment 6

Raphael Bleier, WS 2026

Node v22, PostgreSQL 18 auf Ubuntu. Getestet hab ich mit curl, die Ausgaben stehen unten (Header wie Date/ETag gekürzt). In jedem ex-Ordner liegt zusätzlich eine `postman_collection.json`, die man in Postman importieren kann. node_modules ist nicht dabei, vor dem Start `npm install` und dann `node server.js`.

Exercise 1 - Getting Started

Express installiert mit `npm install`. Content-Type mit `res.setHeader('Content-Type', 'text/html')`. Für den Route-Parameter gibt es `app.get('/:text', ...)`, der Wert steht dann in `req.params.text`.

```
$ curl -i -X GET http://localhost:3000/
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 28

This is a simple application


$ curl -i -X GET http://localhost:3000/mytext
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 45

This is a simple application receiving mytext
```

Exercise 2 - Product List API

products.json wird beim Start einmal mit `fs.readFileSync` eingelesen, mit `JSON.parse` umgewandelt und zur Kontrolle in die Konsole geschrieben. `/products` gibt mit `map` nur die IDs zurück, `/product/:id` sucht mit `find`. Wenn nichts gefunden wird, kommt 404 mit einer Meldung.

```
$ curl -i -X GET http://localhost:3000/products
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 66

{"productIds":["sonyAIV","sony24105","NiconDX16400","tamron1724"]}


$ curl -i -X GET http://localhost:3000/product/sony24105
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 91

{"id":"sony24105","title":"Sony 24-105mm lens","desc":"Best Sony travel zoom","price":1300}


$ curl -i -X GET http://localhost:3000/product/gibtsnicht
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
Content-Length: 42
```

```
{"message":"Product gibtsnicht not found"}
```

Exercise 3 - Database-Backed API

Datenbank importieren

Beim Import mit `psql -f createDB.sql` kam zuerst ein Fehler: `\connect: ... database "webtechProducts" does not exist`. Der Grund: Postgres schreibt Namen ohne Anführungszeichen klein. `CREATE DATABASE webtechProducts` legt also `webtechproducts` an, und das `\connect webtechProducts` danach findet die DB nicht. Ich hab im Skript das \connect auf Kleinbuchstaben geändert, dann ging es. Aus demselben Grund muss in der config.json auch `webtechproducts` stehen, weil der pg-Client den Namen genau so verwendet. Die Warnung zu `default_with_oids` kann man ignorieren, den Parameter gibt es seit Postgres 12 nicht mehr. Kontrolliert hab ich die Tabelle mit `psql -d webtechproducts -c "select * from products"` statt pgAdmin.

Außerdem war die mitgelieferte `config.json` kein gültiges JSON (Komma nach der letzten Klammer), da stürzt `require` ab.

pool.js und config

`new Pool(...)` mit host, user, password und database aus der config. Als host steht bei mir `/var/run/postgresql`, das ist der Unix-Socket. Dann meldet sich Postgres über meinen Linux-User an (peer auth) und ich brauche kein Passwort. Mit normalem Setup wäre das `localhost` plus User und Passwort.

Routen

Alle Queries laufen über `pool.query(text, values)` mit `$1`, `$2` als Platzhalter. Werte direkt in den SQL-String einzubauen wäre eine SQL Injection Lücke. Die Spalte heißt in der DB übrigens `description` und nicht `desc` wie im JSON von Ex2. Der PUT-Handler nimmt `description` aus dem Body, gibt bei leerem Body 400 zurück und bei unbekannter ID 404 (rowCount ist 0). Mit `RETURNING *` bekommt man gleich das geänderte Produkt zurück. Bei DB-Fehlern kommt 500.

```
$ curl -i -X GET http://localhost:3000/products
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 66

{"productIds":["NiconDX16400","sony24105","sonyAIV","tamron1724"]}


$ curl -i -X GET http://localhost:3000/product/sony24105
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 98

{"id":"sony24105","title":"Sony 24-105mm lens","description":"Best Sony travel zoom","price":1300}


$ curl -i -X PUT -H 'Content-Type: application/json' -d '{"description":"Best Sony travel zoom, now on
sale"}' http://localhost:3000/product/sony24105
HTTP/1.1 200 OK
```

```
Content-Type: application/json; charset=utf-8
Content-Length: 111

{"id":"sony24105","title":"Sony 24-105mm lens","description":"Best Sony travel zoom, now on
sale","price":1300}


$ curl -i -X GET http://localhost:3000/product/sony24105
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 111

{"id":"sony24105","title":"Sony 24-105mm lens","description":"Best Sony travel zoom, now on
sale","price":1300}


$ curl -i -X PUT -H 'Content-Type: application/json' -d '{"description":"x"}'
http://localhost:3000/product/gibtsnicht
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
Content-Length: 42

{"message":"Product gibtsnicht not found"}


$ curl -i -X PUT -H 'Content-Type: application/json' -d '{}' http://localhost:3000/product/sony24105
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Content-Length: 49

{"message":"Missing description in request body"}
```

Exercise 4 - Session Authentication

DB `webtechsessions` genauso importiert (gleiches Problem mit Groß/Kleinschreibung). express-session und connect-pg-simple per npm installiert.

Session: `store: new pgSession({ pool, tableName: 'sessions', createTableIfMissing: true })`, `cookie: { maxAge: 1 Stunde }` und `secret: "secret"`. Die Sessions liegen damit in der Tabelle `sessions` in Postgres und nicht im Speicher vom Node-Prozess.

login.js: user und pass aus dem Body und eine parametrisierte Query auf die users Tabelle. Kein Treffer gibt 401 "login failed". Bei Treffer wird `req.session.isAuth = true` und `req.session.username` gesetzt, Antwort ist 200 mit "login successful" und dem Login. express-session schickt dann das Cookie `connect.sid` (HttpOnly) mit.

check_auth.js: Middleware, die schaut ob `req.session.isAuth === true` ist. Wenn ja, kommt `next()` und der eigentliche Route-Handler läuft, sonst 401. Eingehängt als zweiter Parameter, z.B. `app.get("/products", checkAuth, (req, res) => ...)`.

Logout: `req.session.destroy()`. Im Callback wird geprüft, ob `req.session` noch existiert. Wenn nicht, 200, sonst 500.

Test: Ohne Login 401, falsches Passwort 401, mit admin/admin 200 und Cookie. Danach gehen die geschützten Routen, nach dem Logout wieder 401. Dann nochmal einloggen, den Server stoppen und

neu starten: `/products` geht trotzdem, weil die Session in der DB gespeichert ist und das Cookie noch passt.

```
$ curl -i -X GET http://localhost:3000/products
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8
Content-Length: 35

{"message":"Authentication failed"}


$ curl -i -X POST -H 'Content-Type: application/json' -d '{"user":"admin","pass":"falsch"}' -c
cookies.txt -b cookies.txt http://localhost:3000/login
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8
Content-Length: 26

{"message":"login failed"}


$ curl -i -X POST -H 'Content-Type: application/json' -d '{"user":"admin","pass":"admin"}' -c
cookies.txt -b cookies.txt http://localhost:3000/login
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 46
Set-Cookie: connect.sid=s%3A...; Path=/; Expires=Wed, 07 Oct 2026 11:28:28 GMT; HttpOnly

{"message":"login successful","login":"admin"}


$ curl -i -X GET -c cookies.txt -b cookies.txt http://localhost:3000/products
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 66

{"productIds":["NiconDX16400","sony24105","sonyAIV","tamron1724"]}


$ curl -i -X GET -c cookies.txt -b cookies.txt http://localhost:3000/product/tamron1724
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 100

{"id":"tamron1724","title":"Tamron 17-24mm","description":"Full frame 17-14mm 2.8 lens","price":800}


$ curl -i -X GET -c cookies.txt -b cookies.txt http://localhost:3000/logout
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 31

{"message":"Logout successful"}


$ curl -i -X GET -c cookies.txt -b cookies.txt http://localhost:3000/products
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8
Content-Length: 35
```

```
{"message":"Authentication failed"}


$ curl -i -X POST -H 'Content-Type: application/json' -d '{"user":"admin","pass":"admin"}' -c
cookies.txt -b cookies.txt http://localhost:3000/login
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 46
Set-Cookie: connect.sid=s%3A...; Path=/; Expires=Wed, 07 Oct 2026 11:28:29 GMT; HttpOnly

{"message":"login successful","login":"admin"}


### server neu gestartet

$ curl -i -X GET -c cookies.txt -b cookies.txt http://localhost:3000/products
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 66

{"productIds":["NiconDX16400","sony24105","sonyAIV","tamron1724"]}
```